SHIELD CRIMSON

# DevOps & Cloud [SWE] — Class 18 Notes

| | |
|---|---|
| **LECTURE TITLE** | Complete CI/CD & DevSecOps — in practice: **one Flask app deployed four ways** (by hand → Docker → Kubernetes → a 7-job DevSecOps pipeline with CodeQL, pip-audit and Trivy) *(the header title is roughly right, but see the note below about where the time actually went)* |
| **DATE/TIME** | 25 September 2026, 11:00 AM |
| **DURATION** | 120 minutes |
| **INSTRUCTOR** | Nensi Ravaliya |
| **BIG IDEA** | A pipeline is just the manual steps you already know — test, build, push, deploy — written down once, in order, with **security checks wired in as gates** so bad code, bad libraries or a bad image never reach the cluster. |
| **SOURCES** | class transcript + her repo folder for this session, `Nency-Ravaliya/devops-heros` → `session-17-devsecops` (the `demo/` app, its `Dockerfile`, `k8s/` manifests and `.github/workflows/devsecops.yml`, plus the topic READMEs `02-…` to `08-…`) + your own `hey-cicd/` and `hey-cicd-2/` repos + her and your Docker Hub tags (checked 4 Oct). |

## WHAT'S INSIDE


0 ⚡ The 60-Second Version
1 📅 Warm-Up & Logistics
2 🖥️ Meet the App — The DevSecOps Dashboard
3 🧪 Tests, Verbosity & Code Coverage
4 🐋 The Dockerfile Team Quiz
5 ☸️ The Kubernetes Manifests
6 🗺️ One App, Four Deployments — The Plan
7 🏋️ Way 1 — Run It by Hand
8 🏋️ Way 2 — Run It in Docker
9 📤 The Bridge — Pushing Your Image to Docker Hub
10 🏋️ Way 3 — Run It on Kubernetes (Minikube)
11 🛡️ DevSecOps in One Page — SAST, SCA & Image Scanning
12 ⚙️ Way 4 — The DevSecOps Pipeline, Job by Job
13 📡 The Live Run — Watching It Go Green
14 🏋️ Lab — The Whole Pipeline With YOUR Image
15 🔍 Reviewing Your Own Files
16 📑 Cheat-Sheet
17 📝 Homework
18 🗒️ Notes on the Transcript Itself

WATCH OUT

**Read this first — the title is *mostly* honest this time, but the balance isn't.**

The header says **"Complete CI/CD & DevSecOps"**, and DevSecOps *was* taught — SAST (CodeQL), SCA (pip-audit) and image scanning (Trivy) all appear in the pipeline. But they got about **25 minutes** of a 120-minute class, all inside one YAML walkthrough. Roughly **70%** of the session was the same app deployed **by hand, then in Docker, then on Docker Hub, then on Minikube** — a big Docker + Kubernetes revision lap before the pipeline.

Also **not taught**, though they sit in her repo folder for this session: **secret scanning** ( `06-secret-scanning` ), **GHCR** as the registry ( `02-container-registry` — the class used **Docker Hub** instead), and the explicit **security gate** flag `--exit-code 1` ( `07` / `08` ). They're mentioned below only where they fix a gap in what was said (§12.8).

NOTE

**Her session numbering vs. your folder numbering.** In her repo this is `session-17-devsecops` — the Kubernetes manifests even name the app `session17-python` . Your folder is `Class-18` . Go by content, not numbers. Every YAML block, command and expected output below was cross-checked against her real files (and the test run in §7.5 was re-run on 4 Oct: **8 passed, 6 warnings, 69% coverage** — exactly what the class saw).

---

## 0 ⚡ The 60-Second Version

1. **One app, four deployments.** The same Flask "DevSecOps Dashboard" went out **(1)** by hand with `python3 app/app.py` , **(2)** as a Docker container, **(3)** on Minikube via `deployment.yaml` + `service.yaml` , **(4)** through a GitHub Actions pipeline. Each step automates the one before.
2. **Tests come with a score.** `pytest --cov=app` runs the 8 tests *and* reports **code coverage** — what % of your code the tests actually touched. Companies set a threshold (her example: below 60%, it doesn't go to QA or prod).
3. **Kubernetes can't use an image that only lives on your laptop.** So: `docker login` (with a **personal access token**) → `docker tag` (makes a *copy* with your username in front) → `docker push` .
4. **DevSecOps = three new security checks in the pipeline. SAST** reads *your code* (CodeQL). **SCA** checks *the libraries you import* (pip-audit). **Image scanning** checks *the finished container* (Trivy, `--severity HIGH,CRITICAL` ).
5. `needs:` **is the gate.** Docker build runs only if **test + SAST + SCA** all pass. Scan needs build. Push needs scan. Deploy needs push. One failure → everything after it is skipped.
6. `${{ github.sha }}` **is the tag.** Every pipeline run tags the image with the commit's unique ID instead of `latest` , so you always know *exactly* which code is running.
7. **The pipeline deploys to a throwaway cluster.** No EKS? `helm/kind-action` builds a **kind** cluster inside the GitHub runner, `sed` swaps `__IMAGE_TAG__` for the SHA, `kubectl apply` deploys, and `port-forward` + `curl` proves the site works — because you can't open the runner in your browser.
8. **When a pipeline fails silently, turn on debug logging.** *Re-run jobs → Enable debug logging* is the pipeline version of `pytest -vvv` .

---

## 1 🗒️ Warm-Up & Logistics

**A shaky start.** She opened with "*Thursday — today is Friday*" (it was Friday 25 Sep), then found the room setup scrambled:

> CLASS MOMENT
>
> **Class moment.** In Hindi: "*Who came here after the bell? The whole setup has been shaken up.*" → "*I will fix it. Give me 5 minutes.*" A few minutes of projector wrangling later: "*Hello? Okay, can we start now?*"

**The plan for the day, stated up front:**

> SAID IN CLASS
>
> "*I will give you a walkthrough regarding all of the deployment that we are doing. Then one by one we will do it manually. Second we will do it with the Docker. Third thing, same application we are going to deploy with Kubernetes, and fourth way, in all, we are going to deploy our application using the CI/CD pipeline.*"

**Setup you needed before the hands-on:**

| Step | Status in class |
|---|---|
| Create a **new GitHub repo** and copy her `demo/` folder into it | "*Done, right?*" — most had |
| Create a Python **virtual environment** | done |
| Have a **Docker Hub** account (create one if not) | needed for §9 and the final task |
| Minikube installed | from the Kubernetes classes |

**Timing.** At ~12:10 she checked: "*How much time is it? 12:10 — 12:30 that's finished*" for the manual/Docker/K8s part, leaving the pipeline for the last half hour. "*It depends on how fast you run the command.*"

**Mid-class phone call.** Around the Minikube section she took a short call ("*I tried that… that is not working… I will submit the request and let you know*") — not class content.

**The attendance banter (end of class):**

> CLASS MOMENT
>
> Students bargained: "*If the pipeline succeeds — extra attendance?*" She: "*Everyone tells me there are 3 attendances… I will run this and then I will have 100% attendance.*" And when asked how many ran the pipeline with **their own** Docker image: "*Really? Your own image you used? I doubt this.*" Attendance was taken at the very end — "*I know what you have deployed.*"

# 2 🖥️ Meet the App — The DevSecOps Dashboard

SAID IN CLASS

*"Last time, calculator application. This time we created one beautiful application — a dashboard. Complete dashboard… So it's not a hello world application today we are going to deploy."*

CLASS MOMENT

She built the UI with an AI prompt — *"I made a small prompt and made it"* — and was proud of it: *"Look how great I made a website. It's good, right? If I get a client, I'll tell you."*

## 2.1 Why only Flask?

She considered a **Node/React** frontend and dropped it on purpose:

SAID IN CLASS

*"It will be very heavy for you, because then you need to understand all the CI/CD steps for two type of application."*

So: **Flask** serves everything. Frontend = one HTML template + static CSS/JS. Backend = Flask API routes in Python.

PICTURE THIS

**Picture this.** A café where the same person cooks *and* serves. Flask is the cook (the API routes) *and* the waiter (it hands out `index.html`, CSS and JS). One kitchen, one pipeline.

## 2.2 The project layout

```
hey-cicd/                      ← her demo folder, your new repo
├── app/
│   ├── __init__.py
│   ├── app.py                 ← the "triggering file" — runs the app
│   ├── templates/index.html   ← the dashboard page
│   └── static/css/styles.css, static/js/main.js
├── tests/test_app.py          ← 8 unit tests
├── k8s/deployment.yaml
├── k8s/service.yaml
├── .github/workflows/devsecops.yml   ← the pipeline (§12)
├── Dockerfile
├── requirements.txt           ← Flask==3.1.3
├── requirements-dev.txt       ← -r requirements.txt + pytest + pytest-cov
├── pytest.ini                 ← where the tests live
└── .dockerignore              ← empty
```

## 2.3 The routes

She described "four types of GET and POST APIs" — the file actually has **seven routes**:

| Method | Route | What it returns | Shown in class as |
|---|---|---|---|
| GET | `/` | `index.html` — the whole dashboard | *"the main page of my website"* |
| GET | `/health` | `{"status": "healthy", uptime, timestamp}` | *"Is it in a healthy status or not?"* |
| GET | `/api/status` | app name, version `2.0.0`, Python version, uptime, **total requests** | the status cards |
| GET | `/api/greet/<name>` | a random greeting with your name | *"Hi Nancy, may your pipelines always pass"* |
| POST | `/api/add` | `number1 + number2` | the old calculator, kept |
| POST | `/api/calculate` | add / subtract / multiply / divide / power / modulo | the new calculator |
| POST | `/api/pipeline/run` | a **fake** pipeline run with stages and timings | the pipeline simulator |

WHY IT WORKS

**Why the request counter goes up.** Every route calls `_increment_requests()` first. So each time you refresh the status card, `total_requests` ticks 5 → 6 → 7 — she showed exactly that. It's an in-memory variable, so it **resets to 0 whenever the app restarts** (and each of the 2 Kubernetes replicas has its own counter).

TRY IT YOURSELF

**Try it yourself** with `curl` once the app is running (§7):

```
curl http://localhost:5001/health
curl http://localhost:5001/api/greet/Nensi
curl -X POST http://localhost:5001/api/add \
  -H "Content-Type: application/json" -d '{"number1": 10, "number2": 20}'
curl -X POST http://localhost:5001/api/calculate \
  -H "Content-Type: application/json" -d '{"a": 6, "b": 3, "operation": "multiply"}'
```

## 3 🧪 Tests, Verbosity & Code Coverage

### 3.1 The test file

`tests/test_app.py` uses Flask's built-in **test client** — it calls the routes in memory, no server needed.

| # | Test | What it checks |
|---|---|---|
| 1 | `test_home` | `/` returns **200** |
| 2 | `test_health` | `/health` → `status == "healthy"` |
| 3 | `test_greet` | `/api/greet/Nensi` → message contains `Nensi` |
| 4 | `test_add_numbers` | 10 + 20 → `30` |
| 5 | `test_add_numbers_missing_fields` | only `number1` sent → **400** |
| 6 | `test_calculator_multiply` | 4 × 5 → `20` |
| 7 | `test_calculator_divide_by_zero` | 10 ÷ 0 → **400** |
| 8 | `test_status` | `/api/status` → `running`, has `python_version`, `uptime` |

GOOD TO KNOW

**Tests 5 and 7 test the *failure* path.** Good tests don't just check that things work — they check that bad input is rejected with the right error code.

### 3.2 `-v`, `-vv`, `-vvv` — verbose mode

SAID IN CLASS

*"If you are just running `pytest` then you are getting only the output of those tests that you run. If you do `-v` … you are able to see that in this file I run this test and it is passed, in green line. If we are doing `-vv` … for each file what exactly test cases I am running. And if we are doing `-vvv` … all the debug messages."*

SAID IN CLASS

*"It's not only for pytest, by the way. You will see this verbose mode everywhere. Even if you are running your pipeline."* (→ §13.2, the debug-logging checkbox.)

CLARIFIED

**Clarified, since the levels got blurred.** Her point — *more v's, more detail; the idea is universal* — is the one to remember. Precisely, in pytest:

| Flag | What you actually get |
|---|---|
| *(none)* | one dot per test: `........` |
| `-v` | one line per test: `tests/test_app.py::test_home PASSED` |
| `-vv` | same, plus **full, untruncated** assertion diffs when a test fails |
| `-vvv` | a bit more internal detail — **not** CPU/memory usage |

For CPU/memory you'd use other tools (`--durations`, profilers). In her assessment, "*more v = more verbose / debug output*" is the answer.

### 3.3 `requirements.txt` vs `requirements-dev.txt`

```
# requirements.txt          ← what the APP needs to run
Flask==3.1.3

# requirements-dev.txt      ← what YOU need to TEST it
-r requirements.txt
pytest==8.4.2
pytest-cov==6.0.0
```

A student asked whether `-r` meant "recursive". It doesn't:

> SAID IN CLASS
>
> *"No, no. It will suggest you need to fetch the requirements from `requirements.txt`."*

> PICTURE THIS
>
> **Picture this.** `-r requirements.txt` is an `#include`. `requirements-dev.txt` says "*everything on that list, plus pytest and pytest-cov.*" The Docker image installs only `requirements.txt` — testing tools stay out of production.

### 3.4 Code coverage — what `pytest-cov` measures

**Her explanation, via Java/Maven:**

> SAID IN CLASS
>
> *"When we are running maven build… it will generate a target folder and the class files. If we have a test folder then it will test all the test cases… I have 10 files. In one file I get the coverage 67 — means 67% of your code file is correct as per the test cases… second file 70, third 80, 90, 100. At the end they will do average of all the code coverage… I got 60%. That means your code is 60% like having a good code base… and 40% you need to add some more test cases."*

**Where it's used — SonarQube and the threshold:**

> SAID IN CLASS
>
> *"In SonarQube — it's one of the SAST tool, DevSecOps tool… we are using for code coverage. If I'm setting one threshold — if I'm getting 60% or less than 60% code coverage — I'm not going to take this application inside my production, or even QA, staging, anywhere."*

> LOCK THIS IN
>
> **Lock this in.** Coverage = *how much of your code the tests actually ran*. A **threshold** turns it into a gate: below X%, the build stops before QA/staging/production. *"If something will break in the production, we can conquer that issue before even entering production."*

> CLARIFIED

**Clarified, since two ideas got merged.** She said a 60% score means the code is "*good… where no one can hack it.*" That's not what coverage measures:

| | Code coverage | Security scanning (SAST) |
|---|---|---|
| Question | *"Did my tests execute this line?"* | *"Is this line dangerous?"* |
| 100% means | every line ran during tests | — |
| Says anything about hackers? | **No** | **Yes** |

A 100%-covered app can still have an SQL injection; the tests just didn't look for it. Also, SonarQube doesn't *measure* coverage itself — your test tool (pytest-cov, JaCoCo for Java) produces the report and SonarQube **displays and gates** on it, alongside its own static analysis. And the overall % is **total lines covered ÷ total lines**, not a simple average of per-file percentages (a 10-line file at 0% shouldn't drag a 1,000-line file at 90% down to 45%). For her assessment: *coverage = % of code tested, compared against a threshold.*

**This app's real number** (re-run of her exact command):

```
Name              Stmts   Miss  Cover   Missing
-----------------------------------------------
app/__init__.py       0      0   100%
app/app.py          102     32    69%   94, 104-105, 121, 128, 132-133, 145, 179-209, 225, 230, 234
-----------------------------------------------
TOTAL               102     32    69%
======================== 8 passed, 6 warnings ========================
```

TRY IT YOURSELF

**Read the `Missing` column.** Lines **179–209** are the whole pipeline-simulator route — **no test calls `/api/pipeline/run`**. Write a 9th test that POSTs `{"branch": "main", "fail_chance": 0}` and asserts `overall_status == "passed"`, re-run, and watch 69% jump. That's exactly the "*add more test cases*" loop she described.

### 3.5 The small files

| File | What it does | Her note |
|---|---|---|
| `pytest.ini` | `testpaths = tests` (where tests live) · `pythonpath = .` (so `from app.app import app` works) | "*To give the path for my test cases*" |
| `.coverage` | the raw coverage database pytest-cov writes | "*It's generated… this is not needed. It's a cache.*" You can't open it directly. |
| `.dockerignore` | empty | "*Nothing I added… I don't need to ignore anything.*" |
| `path/` (or `.venv/`) | your virtual environment | "*It's created by your virtual environment. I did not create it.*" |
| `SECURITY.md` | GitHub's default security-policy template | not discussed |

GOOD TO KNOW

**Fill `.dockerignore` in your own copy.** Empty means `COPY` *could* pull in your venv, `.coverage` and `.git`. Here it's harmless because the Dockerfile copies only `requirements.txt` and `app/`, but the habit matters: add `path/`, `.venv/`, `.coverage`, `.git/`, `__pycache__/`.

## 4 🐳 The Dockerfile Team Quiz

NOTE

**Deliberate repeat.** The Dockerfile was first taught in **Class 7**. She re-drilled it here because the pipeline builds this exact file — and because, by her face, she suspected half the room had forgotten: "*Docker — remember Docker or not? You forgot?*" Written out in full again below.

She split the room into **Team A, Team B, Team C** and went line by line.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app ./app
EXPOSE 5001
CMD ["python", "app/app.py"]
```

| Line | Asked | Answer given | Verdict / her addition |
|---|---|---|---|
| `FROM python:3.12-slim` | What does it do? | "*Base image.*" | ✅ Pulls the base image. **Image =** `python`, **tag =** `3.12-slim`. |
| `WORKDIR /app` | (team) | creates `/app` | ✅ "*…we need to use this directory whenever I will enter inside my container.*" |
| `COPY requirements.txt .` | (team) | "*Copy requirements.txt inside my container.*" | ✅ Into the current dir, i.e. `/app` |
| `RUN pip install -r requirements.txt` | (team) | "*It will run this command.*" | ✅ "*Simple meaning.*" |
| `COPY app ./app` | (team) | copy the app folder | ✅ "*Copy the app folder inside the app folder.*" |
| `EXPOSE 5001` | "*Anyone?*" | "*It will expose the port.*" | ⚠️ Half right — see below |
| `CMD ["python", "app/app.py"]` | Last team | "*Run our Python application.*" | ✅ `app.py` inside the `app` folder is the triggering file → listens on 5001 |

LOCK THIS IN

`EXPOSE` **doesn't open anything.** Her correction: "*Actually it will not expose the port, but it will tell DevOps people that this is the port number that my application is listening on. So you need to do the* `-p` *port forwarding on this port number.*" `EXPOSE` is documentation; `-p 5001:5001` is the real door.

WHY IT WORKS

**Why** `COPY requirements.txt` **comes before** `COPY app`**.** Layer caching (Class 7). Dependencies change rarely; your code changes constantly. Copying requirements first means `pip install` is a **cached layer** — editing `app.py` rebuilds only the last `COPY`, not the install. In a pipeline that runs on every push, that's minutes saved.

GOOD TO KNOW

**Why the app listens on** `0.0.0.0`**.** The last line of `app.py` is `app.run(host="0.0.0.0", port=5001, debug=True)`. Inside a container, `127.0.0.1` would mean "*only the container itself*" — `0.0.0.0` lets traffic from `-p` reach it (same rule as Class 6). The `debug=True` part comes back in §12.4.

---

## 5 ☸ The Kubernetes Manifests

SAID IN CLASS

*"Hopefully you did not forget the deployment and service. Right?"*

NOTE

**Deliberate repeat** of the Deployment + NodePort Service pattern (Classes 10, 11, 13) — written out in full because the pipeline in §12.10 applies these exact two files.

### 5.1 `k8s/deployment.yaml`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: session17-python
spec:
  replicas: 2
  selector:
    matchLabels:
      app: session17-python
  template:
    metadata:
      labels:
        app: session17-python          # ← must match the selector above
    spec:
      containers:
        - name: session17-python
          image: nensiravaliya28/hey-cicd:__IMAGE_TAG__
          imagePullPolicy: Always       # always pull from Docker Hub
          ports:
            - containerPort: 5001
```

| Field | Value | Why |
|---|---|---|
| `replicas` | `2` | two pods of the dashboard |
| `selector.matchLabels` ↔ `template.labels` | `app: session17-python` | how the Deployment finds its own pods (labels vs selectors again) |
| `image` | `nensiravaliya28/hey-cicd:__IMAGE_TAG__` | `__IMAGE_TAG__` is a **placeholder** the pipeline replaces with the commit SHA (§12.10) |
| `imagePullPolicy: Always` | — | pull fresh from Docker Hub every time; never trust a cached copy |
| `containerPort` | `5001` | matches `EXPOSE 5001` and `app.run(port=5001)` |

**SAID IN CLASS**

*"I know you don't know this `__IMAGE_TAG__`, but I will tell you. This is one of the variable that I'm using… fetching my latest SHA value."*

**WATCH OUT**

**Watch out — `__IMAGE_TAG__` is not a real tag.** If you `kubectl apply` this file by hand without editing it, Kubernetes tries to pull `nensiravaliya28/hey-cicd:__IMAGE_TAG__`, finds nothing, and your pods sit in `ErrImagePull` / `ImagePullBackOff`. That's why she said "*update your image name and then colon and then tag*" before applying. For the manual run (§10) change it to `:latest` (or to your own `yourname/python-web:latest`).

### 5.2 `k8s/service.yaml`

```yaml
apiVersion: v1
kind: Service
metadata:
  name: session17-python
spec:
  type: NodePort
  selector:
    app: session17-python
  ports:
    - port: 80           # the Service's own port inside the cluster
      targetPort: 5001   # the container's port
      nodePort: 30001    # the port on the node, for the outside world
```

**SAID IN CLASS**

*"Inside service.yaml I use a NodePort, and NodePort I given 30001. This is a new NodePort that we are giving. Not the old one, the default one."*

**NOTE**

**The ports again** (Classes 11 and 13 drilled them). Written out once more because §12.10's `port-forward 5001:80` only makes sense with this picture:

```
browser / curl
     |
nodePort 30001   (on the node — Minikube's IP)
     |
port 80          (the Service's ClusterIP)
     |
targetPort 5001  (inside the pod)
     |
containerPort 5001 / Flask on 0.0.0.0:5001
```

## 6 🗺️ One App, Four Deployments — The Plan

```
WAY 1  by hand       python3 app/app.py              → localhost:5001
  │    (you are the pipeline)
  ▼
WAY 2  Docker        docker build / docker run -p    → localhost:5001
  │    (the box is portable)
  ▼
(bridge)  Docker Hub   docker login / tag / push     → image on the internet
  │
  ▼
WAY 3  Kubernetes    kubectl apply + minikube service → 2 pods behind a NodePort
  │    (the box is managed)
  ▼
WAY 4  CI/CD         git push → 7 jobs → kind cluster → curl proves it works
       (nobody types anything)
```

PICTURE THIS

**Picture this.** Learning to drive: first you push the car (Way 1), then you drive it yourself (Way 2), then you hire a driver who keeps two cars ready (Way 3), then you build a self-driving route that checks the brakes, the tyres and the fuel before it ever leaves the garage (Way 4). Way 4 only makes sense because you've done 1–3 with your own hands — which is why she made you.

---

## 7 🏋️ Way 1 — Run It by Hand

### 7.1 The lab

DO THIS YOURSELF

**Do this yourself — Way 1.**

```
cd hey-cicd                          # your repo with her demo folder in it
python3 -m venv .venv                # skip if already created
source .venv/bin/activate            # Windows: .venv\Scripts\activate
pip install -r requirements.txt      # "requirements are already satisfied" if done before
python3 app/app.py                   # starts Flask on 0.0.0.0:5001
```

1. Open **http://localhost:5001**.
2. Click every card (§7.3). Check the terminal: each click logs a line ending in `200`.
3. `Ctrl+C` to stop.

SAID IN CLASS

*"What do we need to do to run my Python application? What is the first step?"* — the class answered `pip install -r requirements.txt`, then *"python3 app/app.py — correct."*

### 7.2 "Port already in use" — `lsof` and `kill`

Her app wouldn't start: something already held **5001**.

```
lsof -i :5001          # who is listening on 5001? → shows the PID
kill <PID>             # stop it
python3 app/app.py     # now it starts — "Debugger is active!"
```

SAID IN CLASS

*"See guys, it's running. 200 means I'm getting the response."*

WATCH OUT

**Do this check before every way.** Her instruction before moving to Docker: *"Do the Ctrl+C. Check `lsof -i` and port number 5001. If you see your application, just use the kill command… Otherwise it will not work whenever you are running inside your Docker container or even the Kubernetes component."* Only one thing can hold a port at a time.

GOOD TO KNOW

**On macOS specifically**, port **5000** is often taken by *AirPlay Receiver* (System Settings → General → AirDrop & Handoff). That's one reason the demo uses 5001.

### 7.3 Tour of the dashboard

| Card | What she did | What happened |
|---|---|---|
| **Status** | refreshed it | request count 5 → 6 → 7 · version, time, total requests update |
| **Greeting** | typed *Nancy*, hit it | *"Hi Nancy! May your pipelines always pass!"* — then again: *"What's up, Nancy! Happy coding!"* (random each time) |
| **Calculator** | `1 + 2` | `3.0` — *"output 3 in decimal"* (the API converts to float) |
| **Pipeline simulator** | branch `main`, failure chance 10%, then **71%** | at 71%, *"Install Deps"* failed after ~2 s and the rest skipped |

CLASS MOMENT

**The modulo story.** *"Last time I added only four operations and one person told me — what if I do 2 modulo 2? So I told them, okay… this time I created complete calculator application."* Now: add, subtract, multiply, divide, **power**, **modulo** — and divide/modulo by zero return a clean **400**.

CLASS MOMENT

**"Is the simulator real?"** Several students tried to read meaning into the fail percentages. She had to repeat it three times: "*It's not an actual pipeline, guys. It's just a simulator… I just added some random stuff. Dummy is right.*"

WHY IT WORKS

**Why 71% almost always dies at stage 1 or 2.** Each of the 8 fake stages independently rolls the dice: `random.random() < fail_chance`. At 0.71, stage 1 fails 71% of the time; reaching stage 2 alive is 29%. Getting through all 8 is $0.29^8 \approx$ **0.005%**. At the default 10%, a full pass is $0.9^8 \approx$ **43%**. The first failure marks every later stage `skipped` — which is exactly how real `needs:` behaves (§12.6).

### 7.4 Who got it running

CLASS MOMENT

She walked the room: "*How many of you deployed this application? Add your name, add your name here.*" Students showed screens; some had "*Build, test and ship securely*" (the dashboard's tagline) up; some were showing the code instead of the site — "*He's showing the code. I'm not asking for the code, show the website.*"

### 7.5 Running the tests with coverage

SAID IN CLASS

"*I made a very good MD file. If there is an award for MD files…*" — and she copied the commands straight out of her README:

```
pip install -r requirements-dev.txt
python3 -m pytest --cov=app --cov-report=term-missing
```

Result in class (and on re-run):

```
tests/test_app.py::test_home ... PASSED   (× 8)
app/app.py   102   32   69%   ...
======== 8 passed, 6 warnings ========
```

SAID IN CLASS

"*See, 8 test cases are passed and 6 warnings I got.*" — and the yellow warning lines in the middle.

WHY IT WORKS

**Where the 6 warnings come from.** All six are the same thing: `DeprecationWarning: datetime.datetime.utcnow() is deprecated`. Python 3.12 deprecated `utcnow()`; the app calls it in `/health` and `/api/status`. Not a failure — but a future Python will remove it. The fix is `datetime.datetime.now(datetime.UTC)`. Warnings are the "medium/low severity" of your test run: not blocking today, a bug tomorrow.

## 8 🏋️ Way 2 — Run It in Docker

DO THIS YOURSELF

**Do this yourself — Way 2.**

```
lsof -i :5001                    # make sure Way 1 is stopped
docker build -t python-web .     # build from the Dockerfile in this folder
docker images                    # → python-web   latest
docker run -it -d -p 5001:5001 --name python-web python-web:latest
docker ps                        # → python-web is Up
```

1. Open **http://localhost:5001** — the same dashboard, now from a container.
2. Test every card. "*Check all of the functionality that we have inside our file.*"
3. Clean up:

```
docker stop python-web
docker rm python-web
docker ps                        # empty
```

1. Refresh the browser — it should **fail** now. If it still loads, your Way-1 app is still running.

**The command, as the class built it together:**

| Part | Class answer |
|---|---|
| Build | "`docker build`, `-t`, *then the name… and then dot*" — "*it will build my Docker image from the Dockerfile*" |
| Image name | `python-web` |
| Check | "`docker images` — *python-web latest*" |
| Run flags | `-it -d -p` — "*port number? 5001, that we mentioned inside our Dockerfile*" |
| Container name | `--name python-web` — "*same container name*" |
| Image | `python-web:latest` — "*and then latest image*" |
| Verify | `docker ps` |

CLASS MOMENT

**Class moment — her terminal fought back.** In Hindi, mid-`docker build`: she'd changed some keyboard setting last time and couldn't remember where — "*What was the number lock? … Writing tools, maybe not*" — then the arrow keys misbehaved, then she was in the wrong directory ("*this is devops-heros, not hey-cicd… where's the directory… reached, reached!*"). Then: "*Which command did we run?* `docker build -t` *— which name?* `python-web` *— and then dot. Okay guys, it's doing.*"

GOOD TO KNOW

`-it` **with** `-d` is harmless but unnecessary for a web server — `-d` alone is enough. `-it` matters when you want an interactive shell.

CLASS MOMENT

She checked screens again row by row and warned the last three stragglers in Hindi: "*Do it — otherwise I'll come over and then you'll have to do it.*" Some answers came back in Portuguese/Spanish ASR noise; nothing was lost.

---

## 9 📤 The Bridge — Pushing Your Image to Docker Hub

### 9.1 Why you need it

SAID IN CLASS

*"You can't use that direct image here in your deployment.yaml. Why? Because it will fetch from Docker Hub. So your image is not in Docker Hub yet."*

PICTURE THIS

**Picture this.** Your laptop is your kitchen; Docker Hub is the shop shelf. Kubernetes is a delivery driver who only picks up from the shelf — it can't walk into your kitchen. (Minikube *can* be pointed at your local images with `minikube image load`, but in every real cluster — and in the pipeline — it's the shelf or nothing.)

CLASS MOMENT

*"I think, like, you never pushed the Docker image in the Docker Hub… I given this in your homework guys somewhere — I have an intuition."*

### 9.2 Three commands: login → tag → push

DO THIS YOURSELF

**Do this yourself — push your image.**

```
# 1. Connect your CLI to Docker Hub
docker login                                    # → "Login Succeeded"

# 2. Make a copy of the image with YOUR username in front
docker tag python-web:latest <your-dockerhub-username>/python-web:latest
docker images                                   # → both names, same IMAGE ID

# 3. Upload it
docker push <your-dockerhub-username>/python-web:latest
```

1. Open hub.docker.com → **My Hub** → **Repositories** → `python-web`, "*one minute ago*".

**Login — use a token, not your password:**

SAID IN CLASS

*"Go to the account setting… create a personal access token."* Later: *"If you push it in Docker Hub then you need a personal access token. Now your Docker Hub password will not work."*

GOOD TO KNOW

Docker Hub's CLI login still accepts a plain password for many accounts, but a **personal access token** (Account settings → Personal access tokens) is the right habit: scoped (read / read-write), revocable, and **required** if you have 2FA or SSO. The pipeline in §12.9 uses one.

**Tag — it's a copy, not a rename:**

SAID IN CLASS

*"It's not renaming the image, it's copy-pasting a new image with another name…* `nensiravaliya28/python-web latest` *and* `python-web latest` *— here is our tagged image."*

SAID IN CLASS

*"This is necessary. You need to put your Docker Hub username, forward slash, and then the name of the image. You can give the same image name or any other image name — let's suppose* `my-app`*. It's your will."*

WHY IT WORKS

**Why the username prefix is mandatory.** An image name is really `registry/namespace/repo:tag`. `python-web:latest` silently means `docker.io/library/python-web:latest` — the **official images** namespace, which you can't write to. Adding `yourname/` points it at your own namespace. And "copy" is generous: `docker images` shows the **same IMAGE ID** twice — it's one image with two labels, no extra disk used.

## 9.3 "Mounted from" — the shared-layer surprise

When she pushed, some lines didn't say `Pushed` — they said **`Mounted from nensiravaliya28/hey-cicd`**.

SAID IN CLASS

*"I have found another concept here. My image name is `python-web` but I can see something mounted from nensiravaliya28/hey-cicd. What is this? Because both are the same images and it's using the same layer… shared layer concept."*

WATCH IT HAPPEN

**Watch it happen — a push, layer by layer.**

```
Frame 1  docker push nensiravaliya28/python-web:latest
Frame 2  Docker asks the registry about each layer's digest (sha256:…)
Frame 3  python:3.12-slim base layers → "Layer already exists" / "Mounted from nensiravaliya28/hey-
cicd"
         (same digest already sits in her other repo → the registry links it, no upload)
Frame 4  her app layers → "Pushed"   (only genuinely new bytes travel)
Frame 5  manifest uploaded → "latest: digest: sha256:… size: …"
```

GOOD TO KNOW

**Your image is real and checkable.** Your `hey-cicd-2/README.md` notes "*this is my pushed docker image anishkanase07/python-web*" — and Docker Hub confirms `anishkanase07/python-web:latest`, pushed **25 Sep, 12:30 IST**, ~49 MB compressed. 🎉 See §15.2 for one thing to know about it (it's **arm64**).

---

## 10 🏋️ Way 3 — Run It on Kubernetes (Minikube)

DO THIS YOURSELF

**Do this yourself — Way 3.**

1. Start the cluster:

```
minikube start
minikube status          # host / kubelet / apiserver: Running
```

1. Edit `k8s/deployment.yaml` → `image:` → replace `__IMAGE_TAG__` (see §5.1). Either hers: `nensiravaliya28/hey-cicd:latest` *(public — "you can use my image")* or yours: `<you>/python-web:latest`.
2. Deploy and check:

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get all          # 2 pods Running, 1 deployment, 1 replicaset, the service
kubectl get pods
```

1. Prove self-healing: `kubectl delete pod <one-pod-name>` → `kubectl get pods` → a new pod with a new name appears (the ReplicaSet keeps it at 2).
2. Open it:

```
minikube service session17-python      # opens the browser on a localhost tunnel
```

1. Clean up (§10.2).

SAID IN CLASS

"*`minikube service` and service name — automatically redirect to localhost and the port that Docker gave you.*"

CLARIFIED

**Clarified — why not `http://<minikube-ip>:30001`?** On macOS/Windows with the Docker driver, Minikube runs inside Docker Desktop's Linux VM, so the node IP isn't reachable from your browser. `minikube service <name>` opens a tunnel and prints a `127.0.0.1:<random>` URL — that's the "*port that Docker gave you.*" (Same fix as Class 13.) On Linux, `$(minikube ip):30001` works directly.

CLASS MOMENT

**"Ignore the third pod."** Her `kubectl get pods` showed an extra pod left from earlier — "*ignore the third pod*" — she deleted it with `kubectl delete pod <name>`. And the ASR caught her slip: "*Docker — not a locker — kubectl delete pod.*"

GOOD TO KNOW

**Couldn't push to Docker Hub?** "*Don't worry — you can use your friend's image, you can use your own image, you can use my image. Anyone's Docker image is enough for you to deploy your K8s deployment.*"

### 10.1 What `kubectl get all` should show

```
NAME                                    READY   STATUS    RESTARTS
pod/session17-python-xxxxxxxxx-aaaaa    1/1     Running   0
pod/session17-python-xxxxxxxxx-bbbbb    1/1     Running   0

NAME                        TYPE        CLUSTER-IP     PORT(S)
service/kubernetes          ClusterIP   10.96.0.1      443/TCP
service/session17-python    NodePort    10.x.x.x       80:30001/TCP

NAME                                READY   UP-TO-DATE   AVAILABLE
deployment.apps/session17-python    2/2     2            2

NAME                                           DESIRED   CURRENT   READY
replicaset.apps/session17-python-xxxxxxxxx     2         2         2
```

*(Shape filled in from the manifests — your hash suffixes and IPs will differ.)* Read the `PORT(S)` column: **`80:30001`** is the Service port and the NodePort from §5.2, side by side.

### 10.2 Clean-up before the pipeline

**SAID IN CLASS**

"*`kubectl delete` your deployment and delete your service… `kubectl delete all --all`. Then minikube stop — you can stop, or you can take it as it is, no problem.*"

**WATCH OUT**

**`kubectl delete all --all` is a sledgehammer.** It deletes **every** pod, deployment, ReplicaSet, service… in the current namespace — including leftovers from other labs you might have wanted. (The built-in `kubernetes` service is recreated automatically, so the cluster survives.) The surgical version:

```
kubectl delete -f k8s/deployment.yaml -f k8s/service.yaml
```

---

## 11 DevSecOps in One Page — SAST, SCA & Image Scanning

Before the YAML, the three new words, together.

**PICTURE THIS**

**Picture this — building a house.**

| Check | House version | Pipeline version | Tool in class |
|---|---|---|---|
| **SAST** | inspect the **blueprint** for design flaws | read **your source code** for dangerous patterns | **CodeQL** (also: SonarQube) |
| **SCA** | check the **bricks you bought** against recall lists | check **third-party libraries** against known-vulnerability lists | **pip-audit** |
| **Image scan** | inspect the **finished house** — wiring, plumbing, everything | scan the **built container** — OS packages, Python, libraries | **Trivy** |

Each one catches something the others can't. Clean code (SAST ✅) built on a vulnerable Flask (SCA ❌) inside an old Debian base image (Trivy ❌) is still a vulnerable app.

| Term | Stands for | Looks at | Runs the app? |
|---|---|---|---|
| **SAST** | **S**tatic **A**pplication **S**ecurity **T**esting | your code | No — reads it |
| **SCA** | **S**oftware **C**omposition **A**nalysis | your dependencies ( `requirements.txt` ) | No |
| **Image scanning** | — | the whole Docker image | No |
| *(DAST — not taught)* | *Dynamic* AST | a *running* app, attacked from outside | **Yes** |

LOCK THIS IN

**Lock this in — "shift left".** Put the cheap checks as early (as far *left* in the pipeline) as possible. A bad library caught by SCA costs you a one-line version bump. The same library caught *after* the image is built, pushed and deployed costs you a rebuild, a re-push and a re-deploy — "*so it will be hectic for you.*"

NOTE

**Also in her repo, not taught:** `06-secret-scanning` (never commit keys — use `os.getenv()` / `${{ secrets.X }}` ; a leaked key must be **revoked and rotated**, deleting the line isn't enough) and `08-security-gates` ("*a scan finds a problem; a gate decides what happens next*"). Worth a read before the next class.

## 12 ⚙️ Way 4 — The DevSecOps Pipeline, Job by Job

SAID IN CLASS

"*Okay guys, now last thing of today session. What we will do in our DevSecOps pipeline. I need everyone here.*"

File: `.github/workflows/devsecops.yml` — name **"Python DevSecOps Pipeline"**.

### 12.1 The triggers

```yaml
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

**SAID IN CLASS**

*"Whenever I'll push inside my main branch it will trigger my CI/CD pipeline. Second thing — on pull request, branches main. So if someone will do the pull request in my main branch, at that time also my pipeline will run. Two conditions we added."*

### 12.2 The shape of the whole thing




**GOOD TO KNOW**

**Every job is a fresh machine.** Each job says `runs-on: ubuntu-latest` and gets its **own** brand-new runner VM. Nothing carries over between jobs unless you pass it explicitly. That's why every job starts with `actions/checkout` — and why the image gets rebuilt in jobs 5 and 6 (§12.8).

LOCK THIS IN

**The first step, always.** "*Always make sure, in CI/CD pipeline, the first instruction is to check out inside your code.*" — "`git clone` *karke uske paas code hoga*" (it `git clone`s, and then the runner has the code).

### 12.3 Job 1 — `test` (Unit Tests)

```yaml
test:
  name: Unit Tests
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Setup Python
      uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - name: Install dependencies
      run: |
        pip install -r requirements-dev.txt
    - name: Run tests
      run: |
        pytest --cov=app --cov-report=term-missing
```

| Step | Her explanation |
|---|---|
| `runs-on: ubuntu-latest` | "*My job will run on Ubuntu latest operating system.*" |
| `uses: actions/checkout@v4` | a **predefined action** — "*checkout means it will enter inside your code base*" |
| `uses: actions/setup-python@v5` | "*You don't need to do the* `pip install` *or* `apt-get install python3` *... You can, by the way. But I have a predefined Python action*" — Python 3.12 in one step |
| `pip install -r requirements-dev.txt` | dev requirements, because tests need pytest |
| `pytest --cov=…` | "*Same command guys that we run manually.*" |

SAID IN CLASS

**On `run: |`** — "*To run a command we have a* `run` *column, and then you can directly mention the command here. Or if you don't want to mention it in one line, you can make a pipe, and after the line you can mention a complete command.*" The `|` keeps every following indented line as a multi-line script.

SAID IN CLASS

**Unit vs integration:** "*Unit test and integration test are the two most popular test cases that we are performing in each microservice.*" This job runs **unit** tests — each route checked in isolation.

GOOD TO KNOW

> **The coverage gate isn't switched on yet.** This job *reports* 69% but never fails on it. To make §3.4's threshold real, add `--cov-fail-under=60`: pytest then exits non-zero below 60% and the job goes red.

### 12.4 Job 2 — `sast` (CodeQL)

```yaml
sast:
  name: SAST – CodeQL
  runs-on: ubuntu-latest
  permissions:
    contents: read
    security-events: write        # allowed to upload findings to the Security tab
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Initialize CodeQL
      uses: github/codeql-action/init@v3
      with:
        languages: python
    - name: Analyze code
      uses: github/codeql-action/analyze@v3
```

> **SAID IN CLASS**
>
> *"Have you heard this CodeQL? CodeQL is one of the predefined action inside GitHub Actions to do the SAST scanning… Let's suppose I used some kind of a function that directly exposes my API, or directly hacker can hack my application — which is a bad way to write a code. You wrote a code, but that is a bad way to write a function, or maybe loop something you added in a wrong manner. It will analyze those things and will give you error."*

**The language line:**

| Your app | `languages:` |
|---|---|
| Python (this one) | `python` |
| Java | `java` (she said `java`; CodeQL's name is `java-kotlin`, `java` still works) |
| Node.js / React | she said `node`; CodeQL's name is **`javascript`** (or `javascript-typescript`) |

**Two steps, two jobs:** `init` = "*initializing your CodeQL*" for one language; `analyze` = "*actually analyzing your code.*"

CLARIFIED

**Clarified, three things about CodeQL.**

1. **It doesn't produce code coverage.** She said "*after analyzing your code it will generate a code coverage.*" It produces **security alerts** (a SARIF report) that appear under the repo's **Security → Code scanning** tab — file, line, rule, severity. Coverage is pytest-cov's job (§3.4).
2. **Findings don't fail the job by default.** CodeQL uploads alerts and the job usually goes green anyway. Blocking on them is a separate setting (branch protection / code-scanning merge rules). So in this pipeline it's an *inspector*, not yet a *gate*.
3. **Private repos need GitHub Advanced Security.** Code scanning is free on **public** repos. On a private repo without GHAS, the `analyze` upload fails. If your copy is private and job 2 goes red, that's why — make the repo public or remove the job.

TRY IT YOURSELF

**A finding to look for.** The last line of `app.py` is `app.run(host="0.0.0.0", port=5001, debug=True)`. CodeQL's Python rules include **"Flask app is run in debug mode"** — the Werkzeug debugger lets anyone who can reach it run code on the server. Exactly the "*directly exposes*" example she gave. After your pipeline runs, open **Security → Code scanning** and see if it's there. The fix: `debug=os.getenv("FLASK_DEBUG") == "1"`.

### 12.5 Job 3 — `sca` (pip-audit)

```yaml
sca:
  name: SCA - Dependency Scan
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Setup Python
      uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - name: Install dependencies
      run: |
        pip install -r requirements.txt
        pip install pip-audit
    - name: Run dependency scan
      run: |
        pip-audit
```

SAID IN CLASS

*"Inside my Python there is one library, pip-audit. It will analyze all the dependencies inside your requirements.txt and it will check — is there any deprecated version?… We will analyze the modules and libraries — can we directly hack these libraries or not. If it is an issue of auditing, then it will fail your pipeline in this stage."*

CLARIFIED

**Clarified, since three words were off.**

| She said | More precisely |
|---|---|
| SCA is a "*security test we are performing dynamically*" | SCA is **static** — it reads package names + versions, never runs them. *Dynamic* testing (DAST) is a different category. |
| it checks for "*deprecated*" versions | it checks for **known vulnerabilities** (published CVEs / advisories in the PyPI + OSV databases). Deprecated ≠ vulnerable: an old version can be safe; a brand-new one can have a CVE. |
| it would check "*pytest — is it deprecated or not*" | this job installs only `requirements.txt` (Flask), so pytest is **not** audited here. That's arguably right — pytest never ships in the image — but know it. |

Her bottom line is correct and is the one to give: "*pip-audit checks the libraries in requirements.txt; if one is vulnerable, the pipeline fails at this stage.*" (Unlike Trivy and CodeQL here, pip-audit **does** exit non-zero on a finding — so this one really is a gate.)

TRY IT YOURSELF

**Run it locally first** — `pip install pip-audit && pip-audit -r requirements.txt`. Then, to *see* a failure, temporarily pin an old Flask in a scratch file ( `Flask==0.12` ) and run `pip-audit -r scratch.txt` — you'll get CVE IDs and the *fixed version* to upgrade to.

### 12.6 `needs:` — why build waits for all three

```yaml
docker-build:
  name: Docker Build
  needs:
    - test
    - sast
    - sca
```

SAID IN CLASS

"*Docker build stage will only run if your all these three stages are successfully passed. If anyone will fail, your complete pipeline will be skipped after the stages.*"

**Why order matters — her argument:**

SAID IN CLASS

"*If we build the Docker image, then push it, then we get to know there is something issue with one of the library in requirements.txt… you need to again do the code changes and then run your CI/CD pipeline. So it will be hectic for you.*"

WATCH IT HAPPEN

**Watch it happen — one red job.**

```
Frame 1  push to main          → test 🟡  sast 🟡  sca 🟡     (all start together)
Frame 2  ~1 min                → test ✅  sast 🟡  sca ❌     (pip-audit found a CVE)
Frame 3  sast finishes         → test ✅  sast ✅  sca ❌
Frame 4  docker-build checks needs → one parent failed → ⏭ skipped
Frame 5  image-scan, push, deploy  → ⏭ skipped ⏭ skipped ⏭ skipped
         Nothing built. Nothing pushed. Nothing deployed. Fix the library, push again.
```

GOOD TO KNOW

**Precisely:** jobs *without* a `needs:` link to the failed one keep running (that's why sast still finishes in Frame 3). Only the jobs **downstream** of the failure are skipped — which here is everything from build onwards.

### 12.7 Job 4 — `docker-build` and `${{ github.sha }}`

```yaml
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Build Docker image
        run: |
          docker build -t session17-python:${{ github.sha }} .
```

SAID IN CLASS

*"Something that you are not aware right now — `github.sha`. It will generate a SHA value at the end of your image. So it will be your tag. This is your image name, colon, and the tag will be SHA and long name. It's not a latest tag."*

**Why not just `latest`?**

SAID IN CLASS

*"Every time whenever I'm building, it will build only the Docker image with the latest tag? No, it's not a good point. Either you need to create one loop where you do the V1, V2, V3 — but you need to enter a programming logic… To reduce those code lines, GitHub introduced this `github.sha` value."*

CLARIFIED

**Clarified — what the SHA actually is.** She described it as *"a random SHA… 256 bits… always unique, whenever you are running your pipeline."* Precisely:

| | Her description | What it really is |
|---|---|---|
| Source | random value generated by GitHub | the **Git commit ID** that triggered the run |
| Size | 256 bits | **SHA-1, 160 bits** — written as **40 hex characters** |
| Unique per | pipeline run | **commit** — re-running the same commit gives the **same** tag |

Proof from her own Docker Hub ( `nensiravaliya28/hey-cicd` ): tags like `c2bc19f15c29d683ef0093da21d16919034e2761` — 40 characters, one per commit she pushed on 24–26 Sep. And that's the *real* superpower: the tag **is** the commit, so `git show c2bc19f` tells you exactly which code is inside that image. `latest` can never tell you that.

WHY IT WORKS

`${{ … }}` **is GitHub's expression syntax**, filled in *before* the shell runs. So the runner actually executes `docker build -t session17-python:c2bc19f1…2761 .` — the shell never sees the braces. Same syntax as `${{ secrets.X }}` from Class 17.

### 12.8 Job 5 — `image-scan` (Trivy)

```yaml
image-scan:
  name: Image Scan - Trivy
  needs:
    - docker-build
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Build Docker image
      run: |
        docker build -t session17-python:${{ github.sha }} .
    - name: Install Trivy
      run: |
        sudo apt-get install -y wget
        wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key \
          | sudo gpg --dearmor -o /usr/share/keyrings/trivy.gpg
        echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] https://aquasecurity.github.io/trivy-repo/deb
generic main" \
          | sudo tee /etc/apt/sources.list.d/trivy.list
        sudo apt-get update && sudo apt-get install -y trivy
    - name: Scan image
      run: |
        trivy image --severity HIGH,CRITICAL session17-python:${{ github.sha }}
```

**Why scan the image at all?**

SAID IN CLASS

*"Let's suppose inside my Dockerfile I mentioned `FROM python:3.12-slim`, and then I build, then I push to Docker Hub, and then I got to know that python 3.12 is deprecated. This is an issue. So before pushing your Docker image you want to check that your image is safe to push." — "Trivy is the most famous tool that we are using to scan our Docker images."*

**Installing it:** "*You can directly search in your Chrome browser — 'install Trivy' — you will see all of the installation steps. Same installation script I just added over here.*" (The script adds Aqua Security's apt repo + signing key, then `apt-get install trivy`.)

**The command:**

SAID IN CLASS

*"You install kubectl, so kubectl is the command. You install minikube, so minikube is the command. Same way, whenever you are installing the Trivy CLI, this is the command — `trivy image`, and then severity."*

| Severity | Her take |
|---|---|
| **CRITICAL** | fetched ✅ |
| **HIGH** | fetched ✅ |
| MEDIUM | *"I don't want to use the low or medium"* |
| LOW | ignored |

SAID IN CLASS

*"There are four types of risks inside your Docker image… If you will not mention this flag — `trivy image` and image name — then it will give you four types. With `--severity HIGH,CRITICAL` — only those two."*

GOOD TO KNOW

Trivy actually has a fifth bucket, **UNKNOWN** (not yet rated). The full list is `UNKNOWN, LOW, MEDIUM, HIGH, CRITICAL`.

CLARIFIED

**Clarified — the most important fix in these notes. As written, this Trivy step never fails.** She said: *"If this stage is passed, that means there is no high or critical severity inside my image. If it is there, then it will fail."* That's the *intent* — but `trivy image` **exits 0 by default even when it finds CRITICALs**. It prints a table and the job goes green. To make it a real gate you need one more flag — which is in **her own repo**, in `07-container-image-scanning/README.md`:

```
trivy image \
  --severity HIGH,CRITICAL \
  --exit-code 1 \
  session17-python:${{ github.sha }}
```

`--exit-code 1` = *"if you find anything at these severities, return 1"* → the step fails → `push` and `deploy` are skipped. In her assessment, give her answer (*"Trivy scans the image for HIGH/CRITICAL vulnerabilities and fails the pipeline"*) — but know that it's `--exit-code 1` that makes the failing happen. Her own repo puts it in one line: **"A scan finds a problem. A gate decides what happens next."**

CLARIFIED

**Clarified — "this is built again… but we don't need to build it again."** She noticed the rebuild and waved it off. It's there because **job 4's runner is gone** — the image died with it (§12.2). Job 6 then builds a **third** time. Consequence: the image you *push* is not byte-for-byte the image you *scanned* (usually identical, but a base-image update between builds could sneak in). The tidy fixes: build → scan → push inside **one** job, or `docker save` the image as an artifact in job 4 and `docker load` it in jobs 5–6.

### 12.9 Job 6 — `push` (Docker Hub)

```yaml
push:
  name: Push Image to Docker Hub
  needs:
    - image-scan
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Login to Docker Hub
      uses: docker/login-action@v3
      with:
        username: nensiravaliya28                    # ← change to YOURS
        password: ${{ secrets.DOCKERHUB_TOKEN }}
    - name: Build and tag image
      run: |
        docker build \
          -t nensiravaliya28/hey-cicd:${{ github.sha }} \
          -t nensiravaliya28/hey-cicd:latest \
          .
    - name: Push image
      run: |
        docker push nensiravaliya28/hey-cicd:${{ github.sha }}
        docker push nensiravaliya28/hey-cicd:latest
```

**Mapping to what you did by hand in §9:**

| Manual (§9) | Pipeline |
|---|---|
| `docker login` | `docker/login-action@v3` — *"I don't need to use the docker login command. You can use it. That's okay. But right now I have my predefined Docker Hub login"* |
| password = PAT | `password: ${{ secrets.DOCKERHUB_TOKEN }}` — *"Inside password you need to mention your Docker Hub token. Not the password."* |
| `docker build` + `docker tag` | one `docker build` with **two** `-t` **flags** → one image, two names |
| `docker push` | two pushes: `:<sha>` and `:latest` — *"Two images I'm pushing, guys."* |

SAID IN CLASS

*"I know this is complicated for you. So you can directly use the normal `docker build` command and then `docker tag` and `docker push` command. Same thing it's doing inside one command."*

**Username — two ways, shown on purpose:**

SAID IN CLASS

*"In username I mentioned my username directly. But what I can do — `secrets.DOCKERHUB_USERNAME` … I give you both type of examples."* The **token** is a secret (*"Docker Hub token — that is the secret name that I created inside my Actions tab. Last time we created one — demo password — same way"*). The username is usually fine in plain text, but a secret works too.

DO THIS YOURSELF

**Do this yourself — the secret.** GitHub repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret** → name `DOCKERHUB_TOKEN`, value = the Docker Hub personal access token (Read & Write). Exactly like `DEMO_SECRET` from Class 17.

WATCH OUT

**Change her username in 4 places, or your run fails here.** *"By the way, you need to enter your Docker Hub username inside CI/CD. It will not push. If you are running the same pipeline it will fail in this stage."* Docker Hub refuses to let your token push into *her* namespace. Edit: `username:` (1), the two `-t` lines (2), the two `docker push` lines (3), **and** `k8s/deployment.yaml`'s `image:` (4) — otherwise job 7 deploys *her* image.

WHY IT WORKS

**Why push both `:sha` and `:latest`?** `:sha` is the permanent, traceable record — never overwritten, so you can roll back to any commit. `:latest` is the convenience pointer that always moves to the newest build — handy for `docker run` by hand, useless for knowing what's deployed.

### 12.10 Job 7 — `deploy` (kind cluster inside the runner)

```yaml
deploy:
  name: Deploy to Kubernetes
  needs:
    - push
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Create k8s Kind Cluster (For testing deployment)
      uses: helm/kind-action@v1.10.0
    - name: Update deployment with new image tag
      run: |
        sed -i "s|__IMAGE_TAG__|${{ github.sha }}|g" k8s/deployment.yaml
    - name: Deploy to Kubernetes
      run: |
        kubectl apply -f k8s/deployment.yaml
        kubectl apply -f k8s/service.yaml
    - name: Check rollout status
      run: |
        kubectl rollout status deployment/session17-python --timeout=60s
    - name: Test running website in CI (Curl)
      run: |
        kubectl port-forward service/session17-python 5001:80 &
        sleep 3
        curl -s http://localhost:5001 | head -n 15
        curl -s http://localhost:5001/api/status
```

Step by step:

| Step | What it does | Her words |
|---|---|---|
| `if: … refs/heads/main && … push` | deploy only on a real push to main — **never on a PR** | *"If someone will do the pull request, at that time they will use it to merge. Right now you can skip it — this is for your reference… standard practice."* |
| `helm/kind-action` | creates a **kind** cluster on the runner | *"Because you don't have an EKS cluster right now… we are going to create a temporary cluster. We call it a kind cluster."* |
| `sed -i "s\|__IMAGE_TAG__\|…\|g"` | swaps the placeholder for this commit's SHA | *"Every time whenever I do `kubectl apply`, it will not use the by-default image, but it will replace that `__IMAGE_TAG__` with my latest SHA value… one of the advanced things."* |
| `kubectl apply -f …` × 2 | deploy + service | *"Already you knew."* |
| `kubectl rollout status … --timeout=60s` | waits until both pods are Ready (or fails after 60 s) | *"Not necessary, but I just added as a best practice — how we are actually using it in the industry."* |
| `port-forward` + `curl` | proves the site actually answers | the clever bit — see below |

**PICTURE THIS**

**What's kind? K**ubernetes **in D**ocker — a whole cluster whose "nodes" are Docker containers. *"Just like Minikube we are using in our local, same thing we can use a kind cluster too."* It's born when the job starts and destroyed when the job ends: perfect for *"does this deploy at all?"*

WATCH IT HAPPEN

**Watch it happen — `sed` rewrites the YAML.**

```
Before (in the repo, forever):
    image: nensiravaliya28/hey-cicd:__IMAGE_TAG__

sed -i "s|__IMAGE_TAG__|c2bc19f15c29d683ef0093da21d16919034e2761|g" k8s/deployment.yaml
       | |            |                                          |
       | |            └── replace with this (the commit SHA)     └── every match on the line
       | └── find this
       └── s = substitute;  | is the separator (instead of /)

After (only on the runner's copy, only for this run):
    image: nensiravaliya28/hey-cicd:c2bc19f15c29d683ef0093da21d16919034e2761
```

The repo file is never changed — the edit lives and dies with the runner.

**The big question she asked the room:**

SAID IN CLASS

*"After running this CI/CD pipeline, I am not able to see my application in my localhost, in my Chrome browser. Why?"* — *"Because the cluster is created inside this CI/CD pipeline runner, and runner is one of the GitHub servers. We can't access this GitHub server."*

SAID IN CLASS

*"It's like an EC2 instance, but you don't know the IP address of that instance — that is the only thing."* — and *"Normally we do port forwarding… `minikube service` we can't use here, because it's not a Minikube, guys, it's a kind cluster. So here we have only one way: port forwarding."*

WATCH IT HAPPEN

**Watch it happen — testing a website nobody can visit.**

```
Frame 1  kind cluster up on the runner; 2 pods Running (rollout status ✅)
Frame 2  kubectl port-forward service/session17-python 5001:80 &
                  runner's localhost:5001  ──►  Service port 80  ──►  pod :5001
         (the & sends it to the background so the next line can run)
Frame 3  sleep 3   — give the tunnel a moment to open
Frame 4  curl -s http://localhost:5001 | head -n 15
         → "Forwarding from 127.0.0.1:5001 -> 5001" / "Handling connection for 5001"
         → <!DOCTYPE html> … DevSecOps Dashboard …        (first 15 lines of index.html)
Frame 5  curl -s http://localhost:5001/api/status
         → {"app": "DevSecOps Dashboard", "status": "running", …}
Frame 6  HTML and JSON are in the job log = the app really works inside the runner ✅
```

CLARIFIED

**Clarified — "port 80".** She said "*I did a port forwarding for my application to the port number 80… GitHub is running on port 80.*" GitHub has nothing to do with 80. In `5001:80`, **5001 is the runner's local port** and **80 is the Service's port** from `service.yaml`, which forwards to `targetPort: 5001` in the pod. Read it as "*local:remote*" — same as `docker run -p`.

TRY IT YOURSELF

**Extend the smoke test** — she said "*you can do curl in all of the APIs… we just tested two.*" Add:

```
curl -s http://localhost:5001/health
curl -s http://localhost:5001/api/greet/Anishka
curl -s -X POST http://localhost:5001/api/calculate \
  -H "Content-Type: application/json" -d '{"a": 2, "b": 10, "operation": "power"}'
```

💡 And add `-f` (`curl -sf`) so an HTTP error **fails** the step instead of just printing it.

### 12.11 The EKS variant (`eks.yaml`)

For students with an AWS account, she pushed a second workflow, `eks.yaml`, to **another branch** ("*I didn't run it, so I put it in another branch*"). It's not in her public repo folder, so this is reconstructed from what she said. **Only the deploy job changes:**

| You need | Why | Stored as |
|---|---|---|
| **AWS access key** | log in to your AWS account | GitHub secret |
| **AWS secret key** | — the other half of the login | GitHub secret |
| **AWS region** (`ap-south-1`, `us-east-1`…) | "*go to that particular region where you want to deploy*" | GitHub secret / variable |
| **EKS cluster name** | "*to access your cluster*" | GitHub secret / variable |

SAID IN CLASS

"*Only four variables you need to add… Then same thing — `kubectl apply` deployment and service — and it will deploy.*" Instead of `kind-action`, the job configures AWS credentials and updates kubeconfig for the named cluster.

GOOD TO KNOW

**The real-world version** of this (in her repo's `03-kubernetes-deployment/README.md`) stores a whole `KUBE_CONFIG` as a secret and uses `kubectl set image` + `kubectl rollout status`. Her repo also notes the honest limit: **a GitHub-hosted runner cannot reach the Minikube on your laptop** — automated CD needs a reachable cloud cluster or a self-hosted runner. That's why the class used kind.

# 13 🎬 The Live Run — Watching It Go Green

## 13.1 The run she showed

She committed and pushed (the commit message on screen: `eks.yaml added`), then narrated the Actions tab — mostly in Hindi:

WATCH IT HAPPEN

**Watch it happen — her pipeline, live.**

```
Frame 1  git add . ; git commit -m "eks.yaml added" ; git push   → Actions tab, zoomed in
         "I ran this pipeline 4–5 times — it failed many times in one stage. I'll show you that
too."
Frame 2  Unit Tests ⟳   SAST – CodeQL ⟳   SCA – Dependency Scan ⟳    — "all three running in
parallel"
Frame 3  all three ✅  → Docker Build starts — "it's checking out… building the image"
Frame 4  Image Scan – Trivy ⟳ — "Trivy is working. See?"
Frame 5  Push Image to Docker Hub ⟳  — her warning: with HER username in the file, YOUR run dies
here
Frame 6  Deploy to Kubernetes ⟳ — "Look, this node is prepared. It's a kind cluster… a test cluster"
         deployment created → service created → rollout ✅
Frame 7  port-forward: "Forwarding from … 5001 -> 5001" · "Handling connection for 5001"
         curl /  → the full DevSecOps Dashboard HTML in the log
         curl /api/status → every API status field
Frame 8  ✅ all 7 green
```

GOOD TO KNOW

**Her earlier attempts are visible on Docker Hub.** `nensiravaliya28/hey-cicd` has **six SHA tags pushed between 23:11 and 23:29 IST on 24 Sep** — that's the "*4–5 times… failed many times*" rehearsal the night before — plus a final `:latest` + SHA on 26 Sep. Each SHA tag = one pipeline run that made it all the way to job 6.

CLASS MOMENT

**Her plan, stated cheerfully:** "*Then we will do the CI/CD pipeline, and that must fail. Then I will tell you how to succeed, and then it must succeed.*" — the failure being the Docker Hub username (§12.9).

## 13.2 Debug logging — `-vvv` for pipelines

At the very end she closed the loop on §3.2:

SAID IN CLASS

*"I was telling this thing whenever you are doing `-v`, verbose mode — see what I did: I just click on Re-run all jobs, then can you see Enable debug logging? If you are running your pipeline with this debug logging, you will see all the debug messages… in violet colour."*

SAID IN CLASS

*"Somewhere your pipeline is failing and you are not getting any kind of error in that stage — then we are using this method. Every CI/CD platform that we are using, they have this verbose mode implemented."*

DO THIS YOURSELF

**Do this yourself.** Actions tab → open any finished run → **Re-run jobs** ▾ → **Re-run all jobs** → tick **Enable debug logging** → Re-run. Expand any step: the extra `##[debug]` lines are the purple ones.

💡 To turn it on for **every** run, add a repository **variable** (or secret) `ACTIONS_STEP_DEBUG = true` (step-level detail) and/or `ACTIONS_RUNNER_DEBUG = true` (runner-level detail).

---

## 14 🏋️ Lab — The Whole Pipeline With YOUR Image

This is the day's final task: *"Once you are able to deploy this application through these four different areas, then in the last task for today you need to just add your Docker Hub ID, and you are going to deploy your own image."*

DO THIS YOURSELF

**Do this yourself — Way 4, end to end.**

1. **Repo.** New **public** GitHub repo (public so CodeQL works — §12.4). Copy *everything* from her `demo/` into it, **including the hidden `.github/` folder** ( `cp -R demo/. my-repo/` — the `/.` copies dotfiles).
2. `.gitignore` your venv and caches: `path/` , `.venv/` , `.coverage` , `__pycache__/` .
3. **Docker Hub token.** hub.docker.com → Account settings → **Personal access tokens** → *Read & Write* → copy it.
4. **GitHub secret.** Repo → Settings → Secrets and variables → Actions → `DOCKERHUB_TOKEN` = that token.
5. **Your username, 4 places** (§12.9): workflow `username:` , both `-t` , both `docker push` , and `k8s/deployment.yaml` → `image: <you>/hey-cicd:__IMAGE_TAG__` (keep the placeholder!).
6. **Push:**

```
git add .
git commit -m "DevSecOps pipeline with my Docker Hub"
git push origin main
```

1. **Watch** the Actions tab: 3 parallel → build → scan → push → deploy.
2. **Verify:** Docker Hub shows `<you>/hey-cicd` with a 40-char SHA tag **and** `latest` ; the deploy log shows the dashboard HTML and the `/api/status` JSON.
3. **Break it on purpose** (one at a time, push, watch, revert): - change `assert data["result"] == 30` to `31` → **test** fails → build/scan/push/deploy **skipped** - delete the `DOCKERHUB_TOKEN` secret → **push** fails → deploy skipped - add `--exit-code 1` to Trivy (§12.8) → see whether `python:3.12-slim` has HIGH/CRITICAL today
4. **Re-run with debug logging** (§13.2) and find the purple lines.

INTERVIEW ALERT

**Interview alert.** "*Walk me through your CI/CD pipeline.*" — the answer is §12.2's diagram, in order: **checkout → unit tests, SAST and SCA in parallel → build → image scan → push with commit-SHA tag → deploy → rollout status → smoke test.** Then the follow-ups they'll ask: *Why the SHA tag?* (traceability / rollback). *What blocks a bad image?* ( `needs:` + a failing exit code — mention `--exit-code 1` ). *Where's the password?* (GitHub secrets, a token not a password).

---

# 15 🔍 Reviewing Your Own Files

## 15.1 `hey-cicd/` — your Class 17 repo

NOTE

This repo now lives in the **Class 17** folder ( `Class-17-CI-CD & GitHub Actions/hey-cicd/` ), where it was built. It's reviewed here too because today's pipeline builds on it.

This folder is your **24 Sep** GitHub Actions practice (10 commits, "*CICD added*" → "*Parallel job*" → "*Secret*" → "*Artifact*"). Your last run — `9c083c2` , *Hello GitHub Actions* — went **green**. Three things worth knowing:

| # | In your file | What happens | Fix |
|---|---|---|---|
| 1 | `on: push: branch: [main, feature/*]` | GitHub didn't reject it — but `branch` **isn't a key it knows**, so the filter is ignored and the workflow fires on a push to **any** branch | `branches:` (plural) — exactly as in her §12.1 file |
| 2 | `build2: needs: build` (commit `6712cf3` ) | no job is called `build` (you have `build1` , `build2` ) → GitHub couldn't parse the file → that run is the red **".github/workflows/cicd.yaml"** one | you commented it out ✅ — the real fix is `needs: build1` , which turns build1 → build2 into a chain |
| 3 | `secrets-demo` (commit `6ac5d07` ) | `runs-on` / `steps` were indented at the same level as the job name → **startup_failure** | you re-indented two spaces in `49bdbea` ✅ — exactly right |

LOCK THIS IN

**What's good.** `secrets-demo` is a nicely written check — `[ -n "$DEMO_SECRET" ]` tests the secret exists **without ever printing it**, and `exit 1` makes the job fail loudly if it's missing. That's the same "fail on purpose" idea as `--exit-code 1` in §12.8. And `build.sh` with `set -e` + a heredoc, uploaded via `actions/upload-artifact` , is clean.

GOOD TO KNOW

`build.sh` still prints "*Session 16 Calculator Application*" — her numbering for that class, harmless.

### 15.2 `hey-cicd-2/` — today's repo

| What's there | Status |
|---|---|
| `README.md` — commit `1c610c2` "*first commit*" (25 Sep, 11:22) | pushed ✅ |
| `README.md` edit: "*this is my pushed docker image anishkanase07/python-web*" | **not committed** ( `git status` → `M README.md` ) |
| `path/to/venv/` | a Python virtual environment |
| her `demo/` code + `.github/workflows/devsecops.yml` | **not in this repo** — your `.code-workspace` opens it from `../../devops-heros/session-17-devsecops/demo` instead |
| GitHub Actions runs on `anishkanase-scaler/hey-cicd-2` | **0** |

So Ways 1–3 happened (from her demo folder, which also has its own `path/to/venv/`), and **your image really is on Docker Hub** — but the pipeline (Way 4) hasn't run from your repo yet. §14 is the to-do.

CLASS MOMENT

`path/to/venv` **— a classic.** Somewhere a tutorial said `python3 -m venv path/to/venv`, meaning "*put your own path here*" — and it was taken literally, creating a folder called `path` containing `to` containing `venv`. Everyone does this once. It works fine; just add `path/` to `.gitignore` so 40 MB of venv never gets committed, or use the conventional `python3 -m venv .venv`.

GOOD TO KNOW

**Your venv is Python 3.14, the Dockerfile is 3.12.** Tests passed on both (re-run 4 Oct on your 3.14 venv: 8 passed, 6 warnings, 69%). Fine for this app — but in general match your local Python to the image's, so "works on my machine" means something.

WATCH OUT

**Your Docker image is `arm64`-only.** Docker Hub lists `anishkanase07/python-web:latest` as **linux/arm64** — because it was built on an Apple-Silicon Mac. It runs on your Minikube (also arm64). But GitHub's `ubuntu-latest` runners, most EKS nodes and most cloud VMs are **amd64**: pulling it there gives `exec format error` and a `CrashLoopBackOff`. (Her own hand-pushed `python-web` is arm64 for the same reason; her **pipeline-built** `hey-cicd` tags are all `amd64` — built on the runner.) To build for both:

```
docker buildx build --platform linux/amd64,linux/arm64 \
  -t anishkanase07/python-web:latest --push .
```

In the §14 pipeline this problem disappears, because the runner builds the image natively as amd64.

# 16 Cheat-Sheet

## Run it by hand

| Command | What it does |
|---|---|
| `python3 -m venv .venv` · `source .venv/bin/activate` | create + enter a virtual environment |
| `pip install -r requirements.txt` | app dependencies (Flask) |
| `pip install -r requirements-dev.txt` | app + test dependencies ( `-r` = include) |
| `python3 app/app.py` | start Flask on `0.0.0.0:5001` |
| `lsof -i :5001` | who's using port 5001 → PID |
| `kill <PID>` | free the port |
| `pytest` / `-v` / `-vv` / `-vvv` | run tests, increasingly verbose |
| `python3 -m pytest --cov=app --cov-report=term-missing` | tests + coverage % + missing line numbers |
| `pytest --cov=app --cov-fail-under=60` | make coverage a gate |

## Docker & Docker Hub

| Command | What it does |
|---|---|
| `docker build -t python-web .` | build from the Dockerfile here |
| `docker images` | list images |
| `docker run -d -p 5001:5001 --name python-web python-web:latest` | run in background, map the port |
| `docker ps` | running containers |
| `docker stop python-web` · `docker rm python-web` | stop · remove |
| `docker login` | connect CLI to Docker Hub (use a PAT) |
| `docker tag python-web:latest <user>/python-web:latest` | add a second name (copy, same IMAGE ID) |
| `docker push <user>/python-web:latest` | upload; watch for "*Mounted from*" |
| `docker buildx build --platform linux/amd64,linux/arm64 -t … --push .` | multi-arch image |

## Kubernetes

| Command | What it does |
|---|---|
| `minikube start` · `minikube status` | start / check the local cluster |
| `kubectl apply -f k8s/deployment.yaml` | create/update the Deployment |
| `kubectl apply -f k8s/service.yaml` | create/update the NodePort Service |
| `kubectl get all` · `kubectl get pods` | see everything / just pods |
| `kubectl delete pod <name>` | prove self-healing |
| `minikube service session17-python` | open the app (tunnel on macOS) |
| `kubectl rollout status deployment/session17-python --timeout=60s` | wait until the rollout finishes |
| `kubectl port-forward service/session17-python 5001:80` | local 5001 → Service 80 → pod 5001 |
| `kubectl delete -f k8s/` | surgical clean-up |
| `kubectl delete all --all` | ⚠️ delete everything in the namespace |
| `minikube stop` | stop the cluster |

## Security tools

| Command | What it does |
|---|---|
| `pip install pip-audit && pip-audit` | SCA — known CVEs in installed packages (exits non-zero on findings) |
| `pip-audit -r requirements.txt` | audit a requirements file |
| `trivy image <image>` | scan an image — all severities |
| `trivy image --severity HIGH,CRITICAL <image>` | only the two worst levels |
| `trivy image --severity HIGH,CRITICAL --exit-code 1 <image>` | …and **fail** if any are found (the gate) |

## GitHub Actions building blocks

| Piece | Meaning |
|---|---|
| `on: push / pull_request: branches: [main]` | triggers ( **`branches`** , plural) |
| `runs-on: ubuntu-latest` | fresh VM per job |
| `uses: actions/checkout@v4` | clone the repo — always step 1 |
| `uses: actions/setup-python@v5` + `python-version: "3.12"` | install Python |
| `run: \|` | multi-line shell script |
| `needs: [test, sast, sca]` | wait for these; skip if any fails |
| `if: github.ref == 'refs/heads/main' && github.event_name == 'push'` | run only on a real push to main |
| `permissions: security-events: write` | let CodeQL upload alerts |
| `uses: github/codeql-action/init@v3` + `analyze@v3` | SAST |
| `uses: docker/login-action@v3` | Docker Hub login |
| `uses: helm/kind-action@v1.10.0` | throwaway Kubernetes cluster on the runner |
| `${{ github.sha }}` | 40-char commit ID → the image tag |
| `${{ secrets.DOCKERHUB_TOKEN }}` | a repo secret |
| `sed -i "s\|__IMAGE_TAG__\|${{ github.sha }}\|g" file` | swap a placeholder |
| Re-run jobs → **Enable debug logging** / `ACTIONS_STEP_DEBUG=true` | pipeline verbose mode |

## Concepts

| Term | One line |
|---|---|
| **Code coverage** | % of your code that tests executed (69% here) |
| **Threshold** | below X%, the build doesn't go to QA/staging/prod |
| **SAST** | static analysis of **your code** — CodeQL, SonarQube |
| **SCA** | known vulnerabilities in **your dependencies** — pip-audit |
| **Image scanning** | vulnerabilities in **the built image** — Trivy |
| **DAST** *(named only)* | attacking the **running** app |
| **Security gate** | a check whose failure **stops** the pipeline |
| **Shift left** | catch problems as early (cheaply) as possible |
| **kind** | Kubernetes in Docker — a disposable test cluster |
| **EKS** | AWS's managed Kubernetes — needs access key, secret key, region, cluster name |
| **Shared layers** | identical layers are stored once; pushes say "*Mounted from*" |
| `EXPOSE` | documents the port; `-p` actually publishes it |

## 17 📝 Homework

- [ ] **1. All four ways, yourself.** Manual → Docker → Kubernetes → pipeline, with **her** image first, then **yours** (§7–§14). "*Better be create from now and you will add it to your name — then it will be like you deploy your name first time.*"
- [ ] **2. Replace the Docker Hub ID** in the workflow (4 places) and `deployment.yaml`, and get **all 7 jobs green** with your own image (§14).
- [ ] **3. Push an image to Docker Hub** if you never did — it was an earlier homework too (§9).
- [ ] **4. Go through the whole codebase** — "*everything that we learned so far.*"
- [ ] **5. Optional — EKS.** If you have an AWS account, try the `eks.yaml` variant (§12.11). Delete the cluster afterwards — EKS bills by the hour.
- [ ] **6. Self-set, recommended:** add `--exit-code 1` to Trivy and `--cov-fail-under=60` to pytest, and write the 9th test for `/api/pipeline/run` (§3.4).

GOOD TO KNOW

**Read before next class** (Terraform, 29 Sep): her repo's `06-secret-scanning` and `08-security-gates` READMEs — short, and they finish the DevSecOps story this class started.

## 18 🗒️ Notes on the Transcript Itself

**The instructor.** The transcript header credits **"Ritesh Prajapati"** — the developer of the *Scaler++ Chrome extension* that downloaded the file, **not** the instructor. This class was taught by **Nensi Ravaliya** (she calls herself "*Nancy*" in the greeting demo; her Docker Hub is `nensiravaliya28`).

**The lecture title.** Header and folder say **"Complete CI/CD & DevSecOps"**. That's fair — SAST/SCA/image scanning were all taught — but it undersells how much of the class was a Docker + Kubernetes revision lap (§6–§10), and it oversells the DevSecOps depth (secret scanning, GHCR and security gates from her repo folder were not covered). A more accurate name would be "*DevSecOps Pipeline — One App, Four Deployments*".

**Transcription quality.** Header shows `Model use: groq, whisper-large-v3-turbo`; English stretches are clean. Damaged stretches:

- **The `docker build` mishap** (~3 min) is in Hindi Devanagari, partly garbled — the gist (keyboard setting, wrong directory, "*devops-heros, not hey-cicd*") is folded into §8.
- **Desk-walking during the Docker and Kubernetes labs** (~10–15 min total) comes out as repeated "*Thank you*", "*Gracias*", "*y y y*", Portuguese "*E aí*" / "*O que é que é isso? Ok, gente*", and a stray "*Carreo*". Pure room noise — skipped.
- **The live pipeline run** is narrated mostly in Hindi (Devanagari transliterating English words — "इस विल फेल", "पहरल", "डॉकर बिल्ड"). Translated and folded into §13.1.
- **The opening** has nonsense ASR ("*You can see the back of the bed… From this bed. Everywhere.*") while she set up the projector — skipped.
- The final word, "*foreign*", is Whisper's usual placeholder for untranscribed speech.

**Folded in rather than dropped:** the Hindi aside about the room setup (§1), "*Abhi hum kya karayenge*" (what we'll do now), "*git clone karke uske paas code hoga*" (§12.2), "*Ye aat ka bane ga tag*" (this will become the tag, §12.7), the threat to the last three stragglers (§8), and "*Trivi se ho raha hai*" (it's happening via Trivy, §13.1).

**Cross-referenced.** Every YAML block, the Dockerfile, the test list, the route table and the README commands come from her `session-17-devsecops/demo`, not the audio. The coverage table and the *6 warnings* explanation come from re-running her exact command on 4 Oct. Docker Hub tags/architectures for `nensiravaliya28/hey-cicd`, `nensiravaliya28/python-web` and `anishkanase07/python-web`, and the GitHub Actions run history for your two repos, were checked via their public APIs. The `kubectl get all` output in §10.1 and the EKS details in §12.11 are **reconstructed** (labelled as such) — the first from the manifests, the second from her description, since `eks.yaml` isn't in her public repo.

**Garbled-term corrections:**

| Transcript says | Actually |
|---|---|
| *"ACICD" / "hey CICD" / "hello CICD"* | **hey-cicd** (the repo / image name) |
| *"CACD" / "CIACD" / "CICD"* | **CI/CD** |
| *"devsacops"* | **DevSecOps** |
| *"Trivi" / "preview"* | **Trivy** |
| *"code QL"* | **CodeQL** |
| *"SASG"* | **SAST** |
| *"SonarCube"* | **SonarQube** |
| *"PyTest-CoV" / "py test coverage"* | **pytest-cov** |
| *"hyphen me" / "verbals mode"* | `-v` / **verbose mode** |
| *"pep install" / "Pip free"* | **pip install** / **pip3** |
| *"requirement.dxt" / ".pxt"* | **requirements.txt** |
| *"python 3 app app.9" / "python ab ab.py"* | `python3 app/app.py` |
| *"lsof-5000" / "50001" / "500123"* | `lsof -i :5001` / **5001** |
| *"300.01"* | NodePort **30001** |
| *"in decibel"* | **in decimal** ( `3.0` ) |
| *"mcravenjal28" / "NancyRavalier28" / "SIE C Ravalia 28" / "NancyRavaja28"* | **nensiravaliya28** (her Docker Hub ID) |
| *"talker hub" / "tokerhub" / "dockerhawk" / "dockerhug"* | **Docker Hub** |
| *"possible access token" / "POSL access token"* | **personal access token** |
| *"shad layer"* | **shared layer** |
| *"Tockele, Stop"* | `docker stop` |
| *"Qtl" / "Cube CTL"* | **kubectl** |
| *"Winniecube" / "mini cube" / "Minikub"* | **Minikube** |
| *"k it is its ke" / "kts"* | `k8s/` |
| *"deployment.tml"* | **deployment.yaml** |
| *"kuburn it is"* | **Kubernetes** |
| *"github.shah" / "Shah value"* | `github.sha` / SHA |
| *"deplicated"* | **deprecated** |
| *"ekase.yaml" / "e-case" / "ekst.yaml"* | **eks.yaml** / **EKS** |
| *"call command" / "cull" / "local command"* | `curl` **command** |
| *"Docker, not a locker"* | her own slip, kept for the laugh |